O Tipo Abstratos de Dados Grafo

- Importante considerar os algoritmos em grafos como **tipos abstratos de dados**.
- Conjunto de operações associado a uma estrutura de dados.
- Independência de implementação para as operações.

Operadores do TAD Grafo

1. *FGVazio(Grafo)*: Cria um grafo vazio.
2. *InsereAresta(V1,V2,Peso, Grafo)*: Insere uma aresta no grafo.
3. *ExisteAresta(V1,V2,Grafo)*: Verifica se existe uma determinada aresta.
4. Obtem a lista de vértices adjacentes a um determinado vértice (tratada a seguir).
5. *RetiraAresta(V1,V2,Peso, Grafo)*: Retira uma aresta do grafo.
6. *LiberaGrafo(Grafo)*: Liberar o espaço ocupado por um grafo.
7. *ImprimeGrafo(Grafo)*: Imprime um grafo.
8. *GrafoTransposto(Grafo,GrafoT)*: Obtém o transposto de um grafo direcionado.
9. *RetiraMin(A)*: Obtém a aresta de menor peso de um grafo com peso nas arestas.

Operação “Obter Lista de Adjacentes”

1. *ListaAdjVazia(v, Grafo)*: retorna *true* se a lista de adjacentes de v está vazia.
2. *PrimeiroListaAdj(v, Grafo)*: retorna o endereço do primeiro vértice na lista de adjacentes de v.
3. *ProxAdj(v, Grafo, u, Peso, Aux, FimListaAdj)*: retorna o vértice u (apontado por *Aux*) da lista de adjacentes de v, bem como o peso da aresta (v, u). Ao retornar, *Aux* aponta para o próximo vértice da lista de adjacentes de v, e *FimListaAdj* retorna *true* se o final da lista de adjacentes foi encontrado.

Implementação da Operação "Obter Lista de Adjacentes"

- É comum encontrar um pseudo comando do tipo:

  for u $\in$ ListaAdjacentes (v) **do** { faz algo com u }

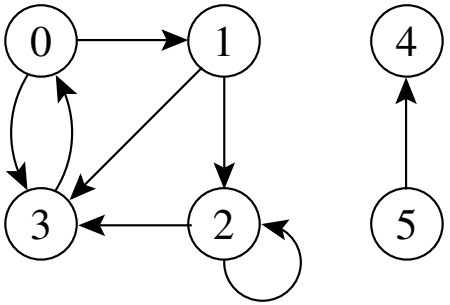
- O trecho de programa abaixo apresenta um possível refinamento do pseudo comando acima.

```
if (!ListaAdjVazia(v,Grafo))
{ Aux = PrimeiroListaAdj(v,Grafo);
  FimListaAdj = FALSE;
  while(!FimListaAdj)
    ProxAdj(&v, Grafo, &u, &Peso, &Aux, &FimListaAdj);
}
```

Matriz de Adjacência

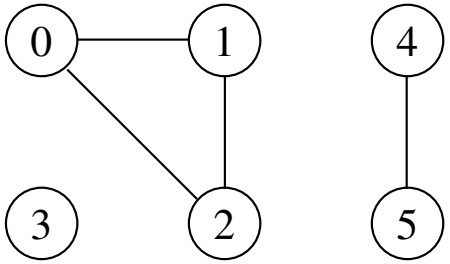
- A matriz de adjacência de um grafo $G = (V, A)$ contendo n vértices é uma matriz $n \times n$ de *bits*, onde $A[i, j]$ é 1 (ou verdadeiro) se e somente se existe um arco do vértice i para o vértice j.
- Para grafos ponderados $A[i, j]$ contém o rótulo ou peso associado com a aresta e, neste caso, a matriz não é de *bits*.
- Se não existir uma aresta de i para j então é necessário utilizar um valor que não possa ser usado como rótulo ou peso.

Matriz de Adjacência: Exemplo



	0	1	2	3	4	5
0		1		1		
1			1	1		
2			1	1		
3	1					
4						
5						

(a)



	0	1	2	3	4	5
0		1	1			
1	1		1			
2	1	1				
3						
4						
5						

(b)

Matriz de Adjacência: Análise

- Deve ser utilizada para grafos **densos**, onde $|A|$ é próximo de $|V|^2$.
- O tempo necessário para acessar um elemento é independente de $|V|$ ou $|A|$.
- É muito útil para algoritmos em que necessitamos saber com rapidez se existe uma aresta ligando dois vértices.
- A maior desvantagem é que a matriz necessita $\Omega(|V|^2)$ de espaço. Ler ou examinar a matriz tem complexidade de tempo $O(|V|^2)$.

Matriz de Adjacência: Estrutura de Dados

- A inserção de um novo vértice ou retirada de um vértice já existente pode ser realizada com custo constante.

```
#define MAXNUMVERTICES  100
#define MAXNUMARESTAS  4500

typedef int TipoValorVertice;
typedef int  TipoPeso;
typedef struct TipoGrafo {
  TipoPeso Mat[MAXNUMVERTICES + 1][MAXNUMVERTICES + 1];
  int NumVertices;
  int NumArestas;
} TipoGrafo;
typedef int  TipoApontador;
```

Matriz de Adjacência: Operadores

```
void FGVazio(TipoGrafo *Grafo)
{ short i, j;
  for (i = 0; i <= Grafo->NumVertices; i++)
    { for (j = 0; j <=Grafo->NumVertices; j++) Grafo->Mat[i][j] = 0; }
}

void InsereAresta(TipoValorVertice *V1, TipoValorVertice *V2,
                  TipoPeso *Peso, TipoGrafo *Grafo)
{ Grafo->Mat[*V1][*V2] = *Peso; }

short ExisteAresta(TipoValorVertice Vertice1,
                   TipoValorVertice Vertice2, TipoGrafo *Grafo)
{ return (Grafo->Mat[Vertice1][Vertice2] > 0); }
```

Matriz de Adjacência: Operadores

```
/* Operadores para obter a lista de adjacentes */
short ListaAdjVazia(TipoValorVertice *Vertice, TipoGrafo *Grafo)
{ TipoApontador Aux = 0;
  short ListaVazia = TRUE;
  while (Aux < Grafo->NumVertices && ListaVazia)
    if (Grafo->Mat[*Vertice][Aux] > 0) ListaVazia = FALSE;
    else Aux++;
  return (ListaVazia == TRUE);
}
```

Matriz de Adjacência: Operadores

```
/* Operadores para obter a lista de adjacentes */
TipoApontador PrimeiroListaAdj (TipoValorVertice *Vertice, TipoGrafo *Grafo)
{ TipoValorVertice Result;
  TipoApontador Aux = 0;
  short ListaVazia = TRUE;
  while (Aux < Grafo->NumVertices && ListaVazia)
    { if (Grafo->Mat[*Vertice][Aux] > 0) { Result = Aux; ListaVazia = FALSE; }
      else Aux++;
    }
  if (Aux == Grafo->NumVertices)
    printf("Erro: Lista adjacencia vazia (PrimeiroListaAdj)\n");
  return Result;
}
```

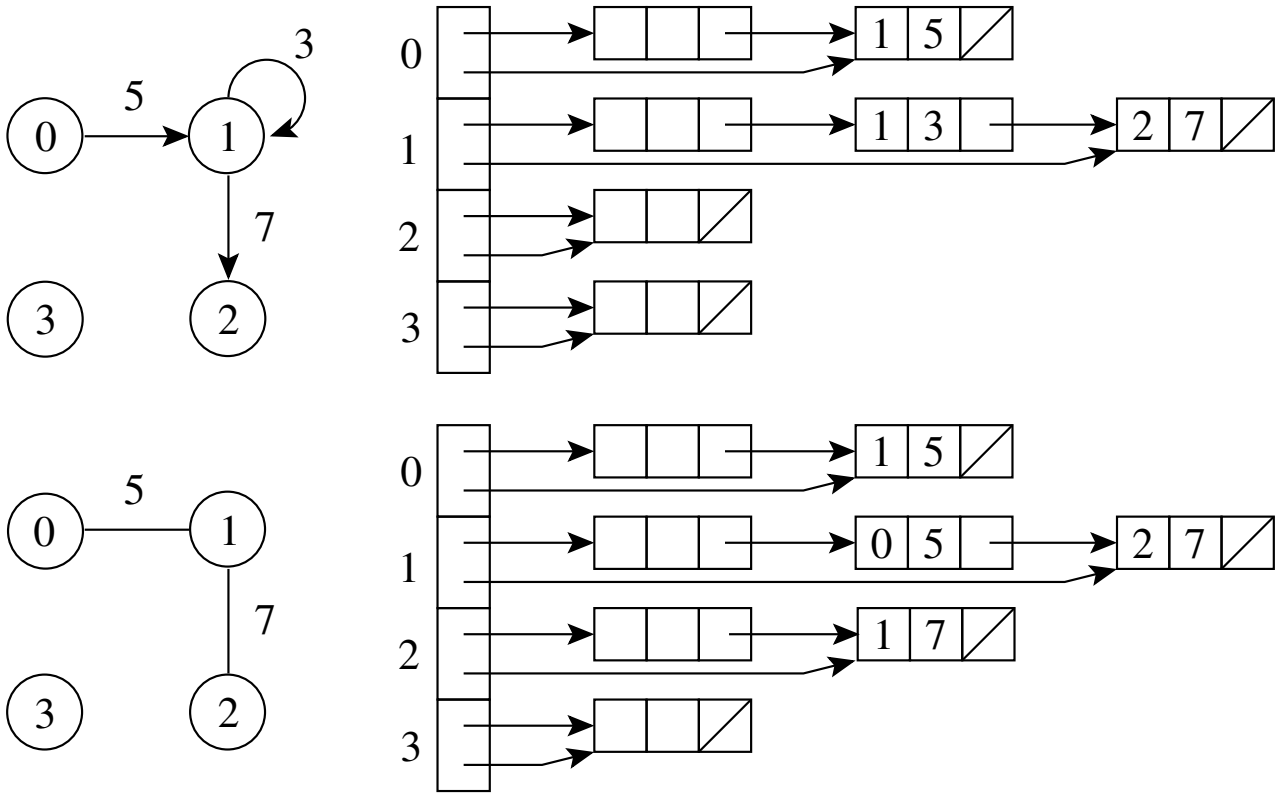
Matriz de Adjacência: Operadores

```
/* Operadores para obter a lista de adjacentes */
void ProxAdj(TipoValorVertice *Vertice, TipoGrafo *Grafo,
             TipoValorVertice *Adj, TipoPeso *Peso,
             TipoApontador *Prox, short *FimListaAdj)
{ /* Retorna Adj apontado por Prox */
  *Adj = *Prox;
  *Peso = Grafo->Mat[*Vertice][*Prox];
  (*Prox)++;
  while (*Prox < Grafo->NumVertices &&
         Grafo->Mat[*Vertice][*Prox] == 0) (*Prox)++;
  if (*Prox == Grafo->NumVertices) *FimListaAdj = TRUE;
}
```

Matriz de Adjacência: Operadores

```
void RetiraAresta(TipoValorVertice *V1, TipoValorVertice *V2,
                  TipoPeso *Peso, TipoGrafo *Grafo)
{ if (Grafo->Mat[*V1][*V2] == 0) printf("Aresta nao existe\n");
  else { *Peso = Grafo->Mat[*V1][*V2]; Grafo->Mat[*V1][*V2] = 0; }
}
void LiberaGrafo(TipoGrafo *Grafo)
{  /* Nao faz nada no caso de matrizes de adjacencia */ }
void ImprimeGrafo(TipoGrafo *Grafo)
{ short i, j; printf("   ");
  for (i = 0; i <= Grafo->NumVertices - 1; i++) printf("%3d", i);
  printf("\n");
  for (i = 0; i <=  Grafo->NumVertices - 1; i++)
    { printf("%3d", i);
      for (j = 0; j <=Grafo->NumVertices - 1; j++)
        printf("%3d", Grafo->Mat[i][j]);
      printf("\n");
    }
}
```

Listas de Adjacência Usando Apontadores



- Um arranjo Adj de $|V|$ listas, uma para cada vértice em V.
- Para cada $u \in V$, $Adj[u]$ contém os vértices adjacentes a u em G.

Listas de Adjacência Usando Apontadores: Análise

- Os vértices de uma lista de adjacência são em geral armazenados em uma ordem arbitrária.
- Possui uma complexidade de espaço $O(|V| + |A|)$
- Indicada para grafos **esparsos**, onde $|A|$ é muito menor do que $|V|^2$.
- É compacta e usualmente utilizada na maioria das aplicações.
- A principal desvantagem é que ela pode ter tempo $O(|V|)$ para determinar se existe uma aresta entre o vértice i e o vértice j, pois podem existir $O(|V|)$ vértices na lista de adjacentes do vértice i.

Listas de Adjacência Usando Apontadores (1)

```
#define MAXNUMVERTICES  100
#define MAXNUMARESTAS  4500
typedef int TipoValorVertice;
typedef int TipoPeso;
typedef struct TipoItem {
  TipoValorVertice Vertice;
  TipoPeso Peso;
} TipoItem;
typedef struct TipoCelula *TipoApontador;
struct TipoCelula {
  TipoItem Item;
  TipoApontador Prox;
} TipoCelula;
```

Listas de Adjacência Usando Apontadores (2)

```
typedef struct TipoLista {
  TipoApontador Primeiro, Ultimo;
} TipoLista;
typedef struct TipoGrafo {
  TipoLista Adj[MAXNUMVERTICES + 1];
  TipoValorVertice NumVertices;
  short NumArestas;
} TipoGrafo;
```

- No uso de apontadores a lista é constituída de células, onde cada célula contém um item da lista e um apontador para a célula seguinte.

Listas de Adjacência Usando Apontadores: Operadores

```
/*-- Entram aqui os operadores FLVazia, Vazia, Insere, Retira e Imprime --*/
/*-- do TAD Lista de Apontadores do Programa 3.4                        --*/
void FGVazio(TipoGrafo *Grafo)
{ long i;
  for (i = 0; i < Grafo->NumVertices; i++) FLVazia(&Grafo->Adj[i]);
}

void InsereAresta(TipoValorVertice *V1, TipoValorVertice *V2,
                  TipoPeso *Peso, TipoGrafo *Grafo)
{ TipoItem x;
  x.Vertice = *V2;
  x.Peso = *Peso;
  Insere(&x, &Grafo->Adj[*V1]);
}
```

Listas de Adjacência usando Apontadores

```
short ExisteAresta(TipoValorVertice Vertice1,
                   TipoValorVertice Vertice2,
                   TipoGrafo *Grafo)
{ TipoApontador Aux;
  short EncontrouAresta = FALSE;
  Aux = Grafo->Adj[Vertice1].Primeiro->Prox;
  while (Aux != NULL && EncontrouAresta == FALSE)
    { if (Vertice2 == Aux->Item.Vertice) EncontrouAresta = TRUE;
      Aux = Aux->Prox;
    }
  return EncontrouAresta;
}
```

Listas de Adjacência Usando Apontadores: Operadores

```
/* Operadores para obter a lista de adjacentes */
short ListaAdjVazia(TipoValorVertice *Vertice, TipoGrafo *Grafo)
{ return (Grafo->Adj[*Vertice].Primeiro == Grafo->Adj[*Vertice].Ultimo);
}

TipoApontador PrimeiroListaAdj(TipoValorVertice *Vertice, TipoGrafo *Grafo)
{ return (Grafo->Adj[*Vertice].Primeiro->Prox); }

void ProxAdj(TipoValorVertice *Vertice, TipoGrafo *Grafo,
             TipoValorVertice *Adj, TipoPeso *Peso,
             TipoApontador *Prox, short *FimListaAdj)
{ /* Retorna Adj e Peso do Item apontado por Prox */
  *Adj = (*Prox)->Item.Vertice;
  *Peso = (*Prox)->Item.Peso;
  *Prox = (*Prox)->Prox;
  if (*Prox == NULL) *FimListaAdj = TRUE;
}
```

Listas de Adjacência Usando Apontadores: Operadores

```
void RetiraAresta(TipoValorVertice *V1, TipoValorVertice *V2,
                  TipoPeso *Peso, TipoGrafo *Grafo)
{ TipoApontador AuxAnterior, Aux;
  short EncontrouAresta = FALSE;
  TipoItem x;
  AuxAnterior = Grafo->Adj[*V1].Primeiro;
  Aux = Grafo->Adj[*V1].Primeiro->Prox;
  while (Aux != NULL && EncontrouAresta == FALSE)
    { if (*V2 == Aux->Item.Vertice)
      { Retira(AuxAnterior, &Grafo->Adj[*V1], &x);
        Grafo->NumArestas--;
        EncontrouAresta = TRUE;
      }
      AuxAnterior = Aux;
      Aux = Aux->Prox;
    }
}
```

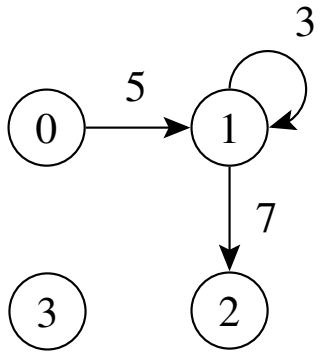
Listas de Adjacência Usando Apontadores: Operadores

```
void LiberaGrafo(TipoGrafo *Grafo)
{ TipoApontador AuxAnterior, Aux;
  for (i = 0; i < GRAfo->NumVertices; i++)
    { Aux = Grafo->Adj[i].Primeiro->Prox;
      free(Grafo->Adj[i].Primeiro);    /*Libera celula cabeca*/
      Grafo->Adj[i].Primeiro=NULL;
      while (Aux != NULL)
        { AuxAnterior = Aux;
          Aux = Aux->Prox;
          free(AuxAnterior);
        }
    }
  Grafo->NumVertices = 0;
}
```

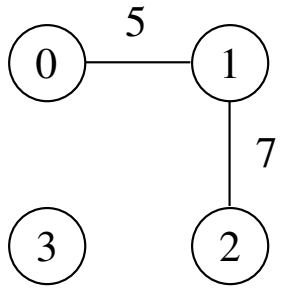
Listas de Adjacência Usando Apontadores: Operadores

```
void ImprimeGrafo(TipoGrafo *Grafo)
{ int i;
  TipoApontador Aux;
  for (i = 0; i < Grafo->NumVertices; i++)
    { printf("Vertice%2d:", i);
      if (!Vazia(Grafo->Adj[i]))
      { Aux = Grafo->Adj[i].Primeiro->Prox;
        while (Aux != NULL)
          { printf("%3d (%d)", Aux->Item.Vertice, Aux->Item.Peso);
            Aux = Aux->Prox;
          }
      }
      putchar('\n');
    }
}
```

Listas de Adjacência Usando Arranjos



		Cab	Prox	Peso
V	0	4	4	
	1	6	5	
	2	2	0	
	3	3	0	
A	4	1	0	5
	5	1	6	3
	6	2	0	7



		Cab	Prox	Peso
V	0	4	4	
	1	6	5	
	2	7	7	
	3	3	0	
A	4	1	0	5
	5	0	6	5
	6	2	0	7
	7	1	0	7

- *Cab*: endereços do último item da lista de adjacentes de cada vértice (nas $|V|$ primeiras posições) e os vértices propriamente ditos (nas $|A|$ últimas posições)

Listas de Adjacência Usando Arranjos

```
#define MAXNUMVERTICES 100
#define MAXNUMARESTAS 4500
#define TRUE 1
#define FALSE 0
#define MAXTAM (MAXNUMVERTICES + MAXNUMARESTAS * 2)

typedef int TipoValorVertice;
typedef int TipoPeso;
typedef int  TipoTam;
typedef struct TipoGrafo {
  TipoTam Cab[MAXTAM + 1];
  TipoTam Prox[MAXTAM + 1];
  TipoTam Peso[MAXTAM + 1];
  TipoTam ProxDisponivel;
  char NumVertices;
  short NumArestas;
} TipoGrafo;
typedef short TipoApontador;
```

Listas de Adjacência Usando Arranjos: Operadores

```
void FGVazio(TipoGrafo *Grafo)
{ short i;
  for (i = 0; i <= Grafo->NumVertices; i++)
    { Grafo->Prox[i] = 0;  Grafo->Cab[i] = i;
      Grafo->ProxDisponivel = Grafo->NumVertices;
    }
}
void InsereAresta(TipoValorVertice *V1, TipoValorVertice *V2,
                  TipoPeso *Peso, TipoGrafo *Grafo)
{ short Pos;
  Pos = Grafo->ProxDisponivel;
  if (Grafo->ProxDisponivel == MAXTAM)
  { printf("nao ha espaco disponivel para a aresta\n"); return; }
  Grafo->ProxDisponivel++;
  Grafo->Prox[Grafo->Cab[*V1]] = Pos;
  Grafo->Cab[Pos] = *V2;  Grafo->Cab[*V1] = Pos;
  Grafo->Prox[Pos] = 0;  Grafo->Peso[Pos] = *Peso;
}
```

Listas de Adjacência Usando Arranjos: Operadores

```
short ExisteAresta(TipoValorVertice Vertice1,
                   TipoValorVertice Vertice2, TipoGrafo *Grafo)
{ TipoApontador Aux;
  short EncontrouAresta = FALSE;
  Aux = Grafo->Prox[Vertice1];
  while (Aux != 0 && EncontrouAresta == FALSE)
    { if (Vertice2 == Grafo->Cab[Aux])
      EncontrouAresta = TRUE;
      Aux = Grafo->Prox[Aux];
    }
  return EncontrouAresta;
}
```

Listas de Adjacência Usando Arranjos: Operadores

```
/* Operadores para obter a lista de adjacentes */
short ListaAdjVazia(TipoValorVertice *Vertice, TipoGrafo *Grafo)
{ return (Grafo->Prox[*Vertice] == 0); }

TipoApontador PrimeiroListaAdj(TipoValorVertice *Vertice,
                               TipoGrafo *Grafo)
{ return (Grafo->Prox[*Vertice]); }

void ProxAdj(TipoValorVertice *Vertice, TipoGrafo *Grafo,
             TipoValorVertice *Adj, TipoPeso *Peso,
             TipoApontador *Prox, short *FimListaAdj)
{ /* Retorna Adj apontado por Prox */
  *Adj = Grafo->Cab[*Prox];  *Peso = Grafo->Peso[*Prox];
  *Prox = Grafo->Prox[*Prox];
  if (*Prox == 0) *FimListaAdj = TRUE;
}
```

Listas de Adjacência Usando Arranjos: Operadores

```
void RetiraAresta(TipoValorVertice *V1, TipoValorVertice *V2,
                  TipoPeso *Peso, TipoGrafo *Grafo)
{ TipoApontador Aux, AuxAnterior;  short EncontrouAresta = FALSE;
  AuxAnterior = *V1;  Aux = Grafo->Prox[*V1];
  while (Aux != 0 && EncontrouAresta == FALSE)
    { if (*V2 == Grafo->Cab[Aux]) EncontrouAresta = TRUE;
      else { AuxAnterior = Aux; Aux = Grafo->Prox[Aux]; }
    }
  if (EncontrouAresta) /* Apenas marca como retirado */
  { Grafo->Cab[Aux] = MAXNUMVERTICES + MAXNUMARESTAS * 2;
  }
  else printf("Aresta nao existe\n");
}
```

Listas de Adjacência Usando Arranjos: Operadores

```
void LiberaGrafo(TipoGrafo *Grafo)
{  /* Nao faz nada no caso de posicoes contiguas */ }

void ImprimeGrafo(TipoGrafo *Grafo)
{ short i, forlim;
  printf("     Cab Prox Peso\n");
  forlim = Grafo->NumVertices + Grafo->NumArestas * 2;
  for (i = 0; i <= forlim - 1; i++)
    printf("%2d:%4d:%4d:%4d\n", i, Grafo->Cab[i],
           Grafo->Prox[i], Grafo->Peso[i]);
}
```